

Joint Cluster-Partition and Orbital-Basis Optimization

A technical note on `cluster_number_decomposition_optimization.py`

August 7, 2026

1 Introduction

Consider a molecular Hamiltonian H in a basis of `norb` spatial orbitals and a fixed-particle-number state $|\psi\rangle$ (typically a DMRG ground state). A cluster number decomposition partitions the orbitals into clusters $C_1, \dots, C_k$, each with total occupation operator $N_{C_i} = \sum_{p \in C_i} n_p$; if $|\psi\rangle$ is close to a joint eigenstate of every N_{C_i} , each acts as an approximate quasisymmetry, enabling the Hilbert-space truncations this codebase is built around. A decomposition is a pair: the partition P , and the orbital basis U in which the clusters are defined, since rotating the orbitals changes which operators the N_{C_i} actually are.

The sister module `cluster_numbers_metrics.py` fixes P by hand and only optimizes U . This module, documented here, searches over both jointly. Exhaustive search over P is impossible – partitioning `norb` orbitals has B_{norb} (Bell number) possibilities, already $B_{16} \approx 10^{10}$ for a modest active space – so it uses a heuristic *beam search* instead: build finer partitions one split at a time, keep a bounded set of promising candidates, and re-optimize U locally after each split.

2 Objects and Notation

Decomposition. A *decomposition* is a pair (P, U) : P a partition of $\{0, \dots, \text{norb} - 1\}$ into clusters, and U the `norb` $\times$ `norb` orbital rotation relative to a fixed reference basis (molecular orbitals, MOs, or natural orbitals, NatOs). U is parameterized by a flat vector $x \in \mathbb{R}^{\binom{\text{norb}}{2}}$: x fills the strict upper triangle of an antisymmetric generator A (in `np.triu_indices(norb, k=1)` order, row-major over $p < q$), and $U = \exp(A)$. Since A is real and antisymmetric, U is always a *proper rotation*, $U \in SO(\text{norb})$: $U^\top U = I$ and $\det U = +1$, never merely orthogonal, never a generic complex unitary. This is the one rotation convention used throughout this module and in the routines it reuses from `src/cluster_number_operators.py`.

RDM data. All cost functions are built from reduced density matrices (RDMs) of $|\psi\rangle$ in the current rotated basis, bundled in an `RDMData` object:

Field	Content
<code>D</code>	spin-summed 1-RDM (always needed)
<code>Gamma</code>	spin-summed 2-RDM (always needed)
<code>rdm3, rdm4</code>	spin-summed 3-/4-RDM (commutator cost only)
<code>h1e, g2e_full</code>	one-/two-electron integrals (commutator cost only)

Cost functional. Every cost function has the same shape: a sum over clusters of a per-cluster cost built from that cluster’s number-operator statistics, $C(P, U) = \sum_{c \in P} c(N_c)$, evaluated at rotation U . Section 3 lists the five supported cost functions.

3 Cost Functions

The variance exponent p raises each cluster’s variance individually before summing, not the total. The extremality cost needs only the 1-RDM: it is a downward parabola in $\langle N_c \rangle$, zero at $\langle N_c \rangle \in \{0, 2|C|\}$ and maximal at half filling; minimizing it pushes every cluster’s occupation toward an integer extreme.

Name	Formula	Needs	Cost
variance	$\sum_C \text{Var}(N_C)^p$	1-,2-RDM	$O(\text{norb}^5)$
extremality	$\sum_C \langle N_C \rangle (2 C - \langle N_C \rangle)$	1-RDM	$O(\text{norb}^2)$
eval_eq	$\sum_C [\text{Var}(N_C) + (\langle N_C \rangle - e_C)^2]$	1-,2-RDM	$O(\text{norb}^5)$
mixed	$c_1 \cdot \text{variance} + c_2 \cdot \text{extremality}$	1-,2-RDM	$O(\text{norb}^5)$
commutator	$\sum_C \langle [H, N_C]^2 \rangle$	1-,2-,3-,4-RDM, h_{1e}, g_{2e}	see text

Table 1: The five cost functions built into this module. $\langle \cdot \rangle = \langle \psi | \cdot | \psi \rangle$ throughout. p is a tunable exponent (default 1); e_C are target integer eigenvalues.

The commutator cost. $F(C) = \langle [H, N_C]^2 \rangle$ measures how far N_C is from commuting with H on $|\psi\rangle$: $F(C) = 0$ means N_C is an exact symmetry generator on this state. Since H and N_C are Hermitian, $[H, N_C]$ is anti-Hermitian, so $F(C) = -\| [H, N_C] |\psi\rangle \|^2 \leq 0$ always; the code returns $-F(C) = \| [H, N_C] |\psi\rangle \|^2 \geq 0$, so that, like the other four costs, smaller is better. Expanding $[H, N_C]$ (a 1-/2-body operator) against another 1-body operator keeps it a 1-/2-body operator, so $F(C)$ contracts the 1-,2-,3-,4-RDM with h_{1e} and g_{2e} ; the full formula is in `src/cluster_number_operators.py`, not reproduced here. Two implementations exist. `number_commutator_cost_v1` recomputes an $O(\text{norb}^8)$ -class tensor contraction independently for every cluster. `number_commutator_cost_v2` observes that every term of $F(C)$ is linear in the cluster indicator $I_C \in \{0, 1\}^{\text{norb}}$, used exactly twice, so $F(C) = I_C^\top K I_C$ is a quadratic form for a single $\text{norb} \times \text{norb}$ kernel K depending only on $H, |\psi\rangle$, not on C ; K is built once per rotation (the same $O(\text{norb}^8)$ -class cost, but paid only once), and every cluster’s score is then a cheap $O(\text{norb}^2)$ bilinear form. Because it needs the 3-/4-RDM at *every* candidate rotation, it is too expensive to drive the search itself – Sections 4–5 always use the cheap variance cost instead – and is reserved for the single polish step (Section 6) on a chosen trajectory endpoint.

4 Cluster Splitting: Fiedler Bisection with a Minimum-Size Floor

Symbol	Meaning
$C, k = C $	the cluster being split, and its size
n_1, n_2	one-/two-body number statistics restricted to C under the current U
W_{pq}	$ \text{Cov}(n_p, n_q) = n_2[p, q] - n_1[p]n_1[q] $, $p \neq q \in C$, $W_{pp} = 0$
L	graph Laplacian of W : $L = \text{diag}(W\mathbf{1}) - W$
v	Fiedler vector: eigenvector of L for its second-smallest eigenvalue
$k_{\min}$	<code>min_child_cluster_size</code> : floor on each resulting child’s size

Orbitals whose occupations fluctuate together end up on the same side: W_{pq} is large exactly when n_p and n_q are strongly correlated, so cutting along the Fiedler vector separates C into two weakly-correlated groups – a cheap proxy for the split that will end up costing the least under the subsequent continuous optimization.

The two fallback branches use different orderings even though both cut at $\lfloor k/2 \rfloor$: the no-correlation branch keeps C ’s own given order, since there is no correlation structure to sort by; the degenerate-Fiedler branch sorts by the Fiedler vector itself, since there is structure, just not one with a clean sign split. Keeping these separate matters for reproducibility of a given run, not for correctness: neither ordering is more “right” than the other in the degenerate case.

The clamp step is the feature this section documents, and is provably a no-op at the default $k_{\min} = 1$: every branch above already produces two non-empty groups, so the natural cut is always already in $[1, k - 1]$, a subset of the clamp range at $k_{\min} = 1$. For $k_{\min} > 1$ the clamp pulls the boundary in from whichever side is too small, moving exactly the Fiedler-boundary-nearest orbitals across – not an arbitrary reshuffling. The $k = 2$ early return is not a fourth case: it only skips a trivial 2×2 eigendecomposition that would return the same split. The Laplacian eigendecomposition costs $O(k^3)$ for a cluster of size k , negligible next to the $O(\text{norb}^5)$ -or-worse cost evaluations elsewhere: Fiedler splitting is never the bottleneck.

Algorithm 1 FIEDLERBIPARTITION($n_1, n_2, C, k_{\min}$)

```
1: if  $k < 2$  then raise error: nothing to split
2: end if
3: if  $k < 2k_{\min}$  then raise error: no split of  $C$  can respect the floor  $k_{\min}$ 
4: end if
5: if  $k = 2$  then return ( $\{C_1\}, \{C_2\}$ ) ▷ trivial; matches what the general path below gives
6: end if
7: compute  $W$ 
8: if  $W \approx 0$  then ▷ no resolvable correlation
9:   order  $\leftarrow C$  in its own given index order;   cut  $\leftarrow \lfloor k/2 \rfloor$ 
10: else
11:   compute Laplacian  $L$  and Fiedler vector  $v$ 
12:   if  $\text{sign}(v)$  is the same for every entry then ▷ degenerate split
13:     order  $\leftarrow C$  sorted by  $v$ , ascending;   cut  $\leftarrow \lfloor k/2 \rfloor$ 
14:   else
15:     order  $\leftarrow C$  sorted by  $v$ , ascending;   cut  $\leftarrow \#\{p : v_p \leq 0\}$ 
16:   end if
17: end if
18: cut  $\leftarrow \min(\max(\text{cut}, k_{\min}), k - k_{\min})$  ▷ clamp; no-op at  $k_{\min} = 1$ 
19: return (order[cut:], order[:cut])
```

5 Beam Search

Symbol	Meaning
(P, U)	a decomposition: partition plus accumulated rotation
B	the beam: a list of up to <code>num_decos</code> decompositions
b, s	<code>num_decos</code> (beam width), <code>num_subdecos</code> (children/parent/round)
k_p, k_c	<code>min_parent_cluster_size</code> , <code>min_child_cluster_size</code>
T	<code>target_num_clusters</code> (optional early-stop target)
traj	the trajectory: best decomposition found for each cluster count
$\text{CANSPLIT}(k)$	$(k > k_p) \wedge (k \geq 2k_c)$: is a cluster of size k worth trying to split

Discrete versus continuous. Which cluster to split is decided by a cheap, cost-function-independent rule: among clusters satisfying `CANSPLIT`, pick those with the largest current $\text{Var}(N_C)$. Only the continuous re-optimization of U after a split, and the ranking used to prune the beam, depend on the user-supplied cost function. This is why the cheap variance cost should drive the search itself: the discrete decisions never depend on which cost function is passed in, but every candidate’s continuous optimization does, so an expensive cost multiplies its cost by the number of candidates touched.

Block-restricted optimization. When cluster C is split, only the entries of the antisymmetric generator A mixing pairs of orbitals *within* C are freed for the subsequent L-BFGS re-optimization; every other entry of A stays zero. This is *not* a cheaper gradient evaluation: the implementation always builds the full $\binom{\text{norb}}{2}$ -length, zero-padded parameter vector and calls $\exp(\cdot)$ on a dense $\text{norb} \times \text{norb}$ matrix regardless of how many entries are free, and reverse-mode automatic differentiation costs about the same multiple of the forward pass independent of how many inputs are free. The real gains are a lower-dimensional optimization landscape (fewer local optima, fewer outer iterations in practice), and, more importantly, an exact structural guarantee. Because A is zero outside C ’s block, $\exp(A)$ is block-diagonal: identity on every orbital outside C . Splitting C therefore leaves every other cluster’s orbitals, and their number-operator statistics, exactly unchanged – the *block-locality invariant* that makes it meaningful to compare candidate splits of different clusters against a common, otherwise-frozen partition. Since A is always real and antisymmetric, $U = \exp(A) \in SO(\text{norb})$ always, and $SO(\text{norb})$ is closed under composition, so every accumulated rotation stays a proper rotation throughout the search.

The search loop. Each child differs from its parent in exactly one cluster: the one chosen for splitting. Every round, every beam member proposes up to s children; all children from all beam members, from every parent, are pooled into one global candidate list, and the best b by cost become the new beam – not the best s per parent kept

Algorithm 2 RUNDECOMPOSITIONOPTIMIZER

```
1:  $B \leftarrow b$  copies of the trivial 1-cluster decomposition, seeded from reference bases (MOs and/or NatOs, cycled, optionally
   each Fiedler-reordered)
2:  $\text{traj} \leftarrow [\arg \min_{\text{cost}} B]$ 
3: while true do
4:   if  $T$  set and every  $(P, U) \in B$  has  $|P| \geq T$  then break
5:   end if
6:   if no cluster in any  $(P, U) \in B$  satisfies CANSPLIT then break
7:   end if
8:   if round-count cap reached then break
9:   end if
10:  candidates  $\leftarrow \emptyset$ 
11:  for each  $(P, U) \in B$  do
12:    rank  $P$ 's clusters satisfying CANSPLIT by current  $\text{Var}(N_C)$ , descending
13:    for each of the top  $s$  such clusters  $C$  do
14:       $(V_1, V_2) \leftarrow \text{FIEDLERBIPARTITION}(n_1, n_2, C, k_c)$  ▷ Algorithm 1
15:       $P' \leftarrow (P \setminus \{C\}) \cup \{V_1, V_2\}$  ▷ only  $C$  changes
16:      re-optimize  $U$ , restricted to  $C$ 's internal generator entries, warm-started at 0
17:      candidates  $\leftarrow \text{candidates} \cup \{(P', U')\}$ 
18:    end for
19:  end for
20:  if candidates =  $\emptyset$  then break ▷ provably unreachable if line 5 did not fire
21:  end if
22:   $B \leftarrow$  best  $b$  of candidates, pooled globally across all parents, by cost
23:   $\text{traj} \leftarrow \text{traj} + [\arg \min_{\text{cost}} B]$ 
24: end while
25: return traj
```

separately. This global pooling means a single strong parent can, in principle, supply all b survivors, at the cost of losing lineage diversity, but it also guarantees the beam's best cost is non-increasing round over round. Because every surviving beam member gains exactly one cluster per round, starting from a uniform 1-cluster beam, every beam member has the same cluster count after any given number of rounds; by induction, the trajectory has no gaps and no duplicates: $\text{traj}[i]$ has exactly $i + 1$ clusters. The restricted re-optimization is warm-started at the all-zero parameter vector – the raw Fiedler cut, with no further rotation – which is a valid point of the restricted search space, so the optimized cost is guaranteed no worse than the cost at that starting point.

Stopping and the trajectory. The loop stops when (1) a requested target cluster count is reached; (2) no cluster anywhere in the beam satisfies CANSPLIT; (3) a round-count safety cap is hit; or (4), defensively, no children were produced in a round. Condition (4) is provably redundant with (2): a beam member contributes at least one child whenever it has a splittable cluster, so if (2) had not already fired, (4) cannot fire either – it is kept only for defensive symmetry against future changes to the eligibility rule. The result is a *trajectory*: the best decomposition found for each cluster count reached, not a single winner. This is necessary because raw cost, on its own, trivially favors finer partitions. The clearest instance is an exact operator identity, not an empirical observation: for the un-split, single all-orbitals cluster, $\text{Var}(N_{\text{total}}) = 0$ for *every* rotation U , because total particle number is invariant under any single-particle basis change and the state has fixed particle number – no optimization is needed to see this. Picking a useful point on the trajectory therefore requires an external metric; Section 7's convergence analysis is that metric in practice, and `best_decomposition(traj, num_clusters=k)` retrieves the trajectory entry with exactly k clusters once a preferred k is known. Every candidate partition, at every round, is validated – covers every orbital exactly once, no empty clusters – before it is accepted into the beam.

6 Polishing

After the beam search terminates, each trajectory entry can optionally undergo one further, full joint reoptimization: all $\binom{\text{nor}^b}{2}$ generator entries are freed at once (not block-restricted), starting from the beam search's own U , for that entry's now-fixed partition. This polishes away suboptimality the greedy, block-restricted process could not reach on its own. Polishing runs once per trajectory entry, entirely independently of the others: it does not feed back into how the trajectory was built. As with each split, it is warm-started at the beam search's own rotation, so

the polished cost is guaranteed no worse than the unpolished one – and it is the natural place to switch to a more expensive, more physically faithful cost such as the commutator cost, since it now runs only once per entry rather than once per beam-search candidate.

7 Downstream Sector Analysis

Selected trajectory entries – by default, every entry with at least two clusters, since a single all-orbitals cluster carries no symmetry information – are passed through the same K-sectors/K-states Hilbert-space-truncation convergence analysis used by `cluster_numbers_metrics.py`’s fixed-partition workflow, reused essentially unchanged: for each entry and each requested maximum electron-transfer count, it ranks symmetry sectors (respectively, decoupled eigenstates) by proximity to the reference state, and reports how the projected energy and retained dimension converge as more are kept, together with whether chemical accuracy is reached. The only structural difference is bookkeeping: there, one `cluster_matrix` is shared across several bases, so sectors are computed once; here, every entry carries its own partition, so sectors are rebuilt per entry. This is the practical answer to Section 5’s open question of which trajectory entry to prefer: the one reaching chemical accuracy with fewest retained sectors or states, not the one with lowest raw cost.

8 Usage

```
python cluster_number_decomposition_optimization.py h2o sto-3g 2.0 variance \
  --bond-angle 104.5 --initial-basis both --target-num-clusters 3
```

Flag	Meaning
<code>--num-decos,</code> <code>--num-subdecos</code>	beam width b , children per parent per round s (default 4, 4)
<code>--min-parent-cluster-size</code>	k_p : clusters at or below this size are never chosen for splitting (default 1)
<code>--min-child-cluster-size</code>	k_c : neither child of a split may end up smaller than this (default 1)
<code>--target-num-clusters</code>	stop once this many clusters is reached (default: run to singletons)
<code>--initial-basis</code> {MOs,NatOs,both}	reference basis (or bases) seeding the beam (default MOs)
<code>--fiedler-reorder</code>	reorder each initial basis by its own global Fiedler vector
<code>--no-polish,</code> <code>--polish-maxiter</code>	skip, or tune the iteration budget of, Section 6’s polish step
<code>--analyze-num-clusters,</code> <code>--skip-sector-analysis,</code> <code>--skip-K-sectors,</code> <code>--skip-K-states</code>	which trajectory entries get Section 7’s analysis, and which half of it

DMRG, HPC, logging and output-path flags (`--bond-dim`, `--n-sweeps`, `--n-threads`, `--output-dir`, ...) are shared verbatim with `cluster_numbers_metrics.py`; see `--help` for the full list.

As a library, the search can be driven directly on RDMs computed elsewhere:

```
from cluster_number_decomposition_optimization import (
    RDMDData, DecompositionOptimizerConfig, run_decomposition_optimizer,
    make_variance_cost_constructor, polish_decomposition, best_decomposition,
)
rdm_data = RDMDData(D=D, Gamma=Gamma)
config = DecompositionOptimizerConfig(num_decos=4, num_subdecos=6, min_parent_cluster_size=2)
traj = run_decomposition_optimizer(make_variance_cost_constructor(), rdm_data, config)
deco = best_decomposition(traj, num_clusters=3)
deco = polish_decomposition(deco, rdm_data, make_variance_cost_constructor())
```